Arquitectura del Microcontrolador I

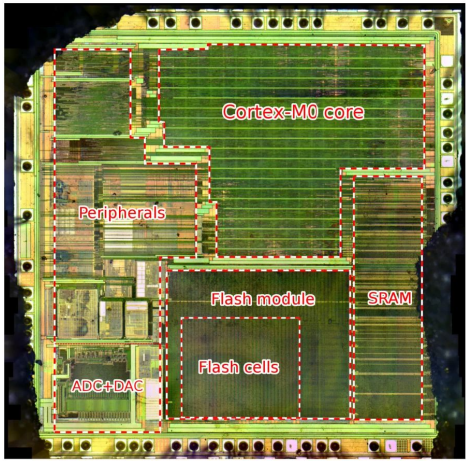
Temas

- Parte I
  - Introducción a la arquitectura
  - Procesador Cortex
  - Registros
  - Set de Instrucciones
  - Modelo de memoria

Introducción - Arquitectura

¿Qué es y por qué conocerla?

- La “arquitectura” es una descripción de las estructuras que componen un sistema y su interacción.
- Se describe desde distintos niveles de abstracción y desde distintas “vistas” que en conjunto describen el sistema completo.
- Conocer la arquitectura permite:
  - Fase inicial: Tener los conocimientos básicos para trabajar con la herramienta
  - Fase avanzada: Obtener la mejor performance y depurar casos problemáticos



¿Por qué importa?

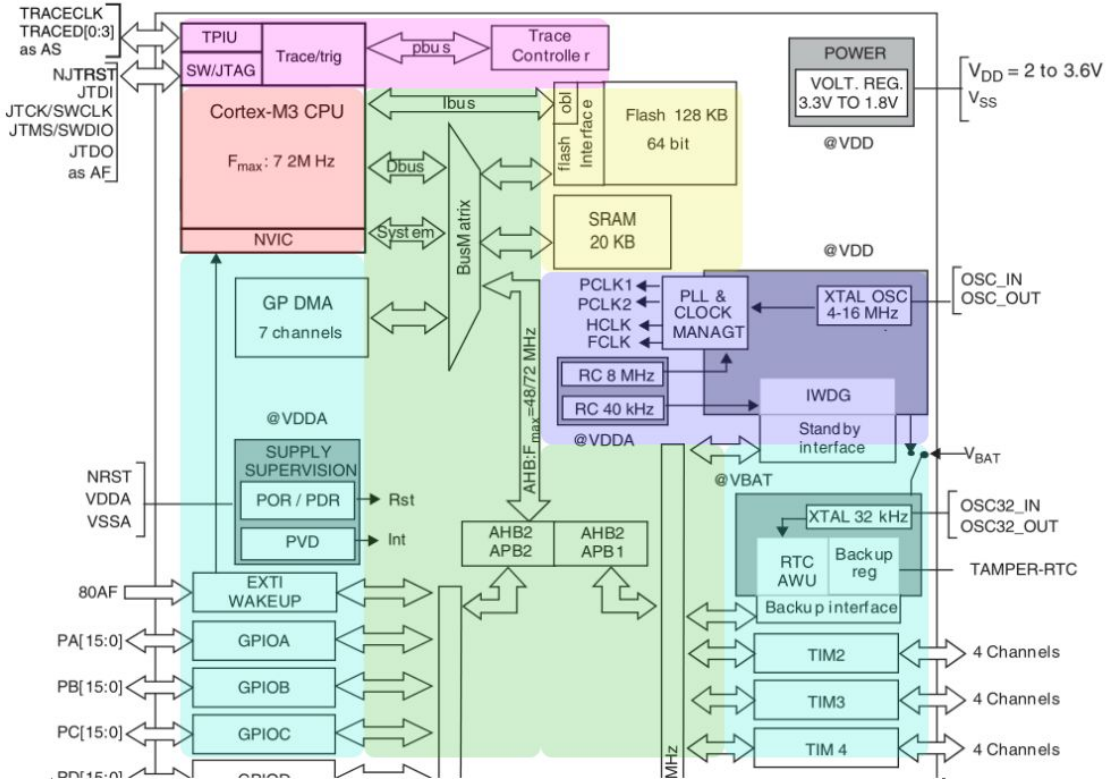
- **Procesador**
  - ¿Qué capacidades tiene mi µC?¿Hasta qué frecuencia de clock soporta?¿cuánto le toma ejecutar instrucciones?¿Puede trabajar con números reales?¿Es el más conveniente para aplicaciones de tiempo real?
- **Manejo de Excepciones**
  - Respuesta a eventos, posibilidad de implementar aplicaciones de tiempo real, velocidad de atención
- **Memoria**
  - Tamaño del programa que puedo cargar, tamaño de datos permanentes (elementos pre-programados para interfaces de usuario) y volátiles (procesamiento de señales)
- **Características de Debug**
  - El 90% del tiempo se utiliza: ¿qué información de debug puedo obtener? ¿Cómo? ¿A qué costo?
- **Buses**
  - ¿Limitación de velocidad de acceso? ¿Uso de periférico DMA?
- **Alimentación**
  - Características de bajo consumo compatibles con el proyecto, facilidades en el manejo de baterías
- **Periféricos**
  - ¿Qué puede hacer?

Capas de la arquitectura

Capa	Microprocesador	+Microcontrolador
Set de instrucciones	• Modelo de programación • Excepciones • Modelo de memoria • Arquitectura del set de Instrucciones (ISA)	• Estrategia de acceso a periféricos
Organización	• Registros del procesador • Estructura del procesador • Buses • “Periféricos” del procesador	• Complementos de manejo de Interrupciones • Estructura • Buses • Periféricos • Acceso a memoria • Reloj
Tecnología	• Implementación micro(nano)electrónica	• Implementación de la memoria • Características eléctricas

Caso de estudio: STM32F103C8

- Procesador ARM Cortex M3
- Manejo de Excepciones
- Memoria
- Características de Debug
- Buses
- Temporización
- Alimentación
- Periféricos

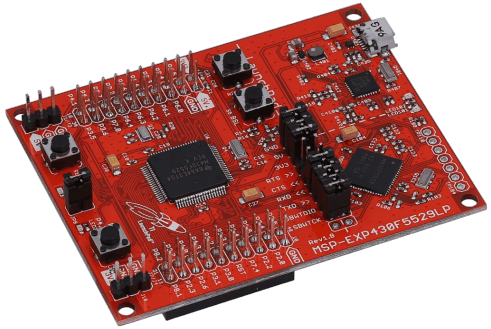


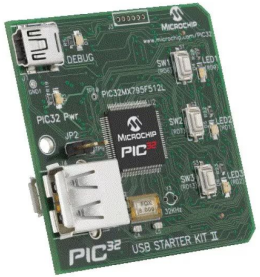
Procesador: Cortex de **ARM**

- ARM diseña micro**procesadores** y los licencia (no fabrica)
  - Por un lado, diseña arquitecturas de set de instrucciones, como la ARMv7-M
  - Por otro lado, diseña procesadores físicos o “cores” que pueden fabricarse
- ST le compra el diseño a ARM, le agrega buses y periféricos y así crea sus µCs
  - El diseño del procesador del **STM32F103** es un **Cortex-M3** de ARM
  - Por lo tanto, al describir la arquitectura, hablaremos en parte de los **Cortex-M3** y habrá parte de la documentación que corresponderá a los Cortex
- El desarrollo para un microcontrolador con core ARM tiene cierta “portabilidad” entre plataformas

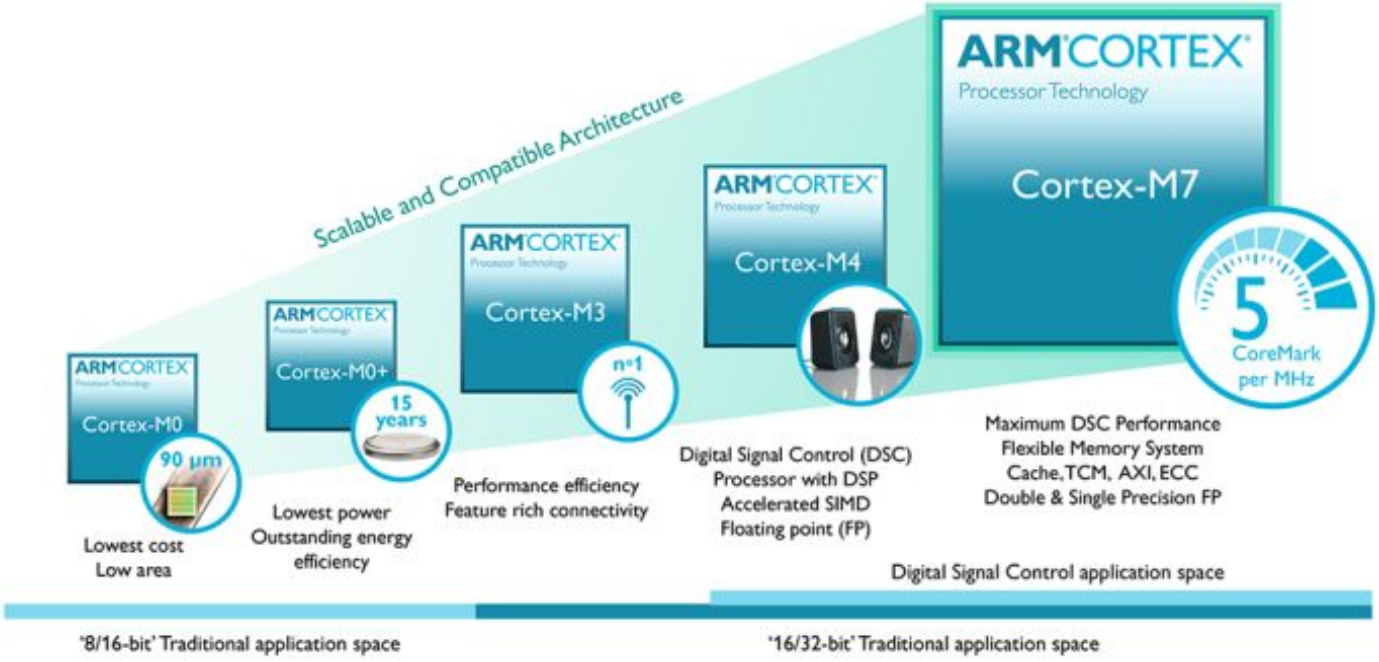
ARM y otras arquitecturas

- Otros µCs
  - Basados en Cortex de ARM
    - STM32 (ST)
    - AVR32 (ATMEL)
    - MSP432 (TI)
    - LPC1700 (NXP)
  - Que no están basados en Cortex
    - ESP32 (Espressif)
    - PIC32 (Microchip)
    - DSPIC (Microchip)
    - C2000 (TI)
    - MSP430 (TI)
    - Basados en RISC-V





“Categoría” dentro de la familia Cortex



http://www.jkelec.co.kr/img/lecture/arm_arch/arm_arch_3.html

Métricas sobre la arquitectura

¿Cómo comparamos?

- Velocidad del procesador
  - Es lo primero que pensamos pero no es tan fácil de evaluar, tiene diferentes dimensiones

$$\tau_{CPU}\left[\frac{s}{tarea}\right] = N\left[\frac{instrucciones}{tarea}\right] \times CPI\left[\frac{ciclos}{instrucción}\right] \times \frac{1}{f_{clock}}\left[\frac{segundos}{ciclos}\right]$$

- Eficiencia energética
  - ¿con qué costo energético se logra?
- Buses y memoria
  - Más allá del tamaño disponible, pueden influir en la velocidad de operación dependiendo del tamaño de código
- Aptitud para tiempo real
  - Velocidad absoluta de respuesta
  - Determinismo en la respuesta
- Aplicaciones específicas
  - Comparar dentro de una familia de aplicaciones

- Existen distintos **“benchmarks”**
  - Son en general conjuntos de programas que se hace correr al procesador en ciertas condiciones controladas
- Cada benchmark está pensado para probar aplicaciones específicas
  - Se busca el adecuado para la aplicación
  - Pero hay una relación de compromiso: cuanto más antiguo y genérico, hay una mayor base de comparación

Benchmark histórico

- Histórico y vigente por la disponibilidad de evaluaciones: Dhrystone
- Mide “cuánto puede hacer” un sistema vs la computadora VAX 11/780 (la primera computadora de 1 Mega instrucción por segundo)
- Ejecutaba 1757 ciclos del test de Dhrystone por segundo
- Si un procesador ejecuta **N**∗1757 veces el test en 1s es una máquina de **N** DMIPS, y si lo hace con un clock de **M** MHz, es de **N/M** DMIPS/MHz

Ejemplo: Tomo el SMT32f103C8 y corro el código del test a 72 MHz. Me da que corrió 9066120 veces en un minuto. Calculo los dhrystone por segundo: 9066120/60 = 151102

Luego, lo normalizo para obtener DMIPS: 151102/1757 = 86

Finalmente calculo DMIPS por Hz: 86/72 = **1.194** DMIPS/MHz

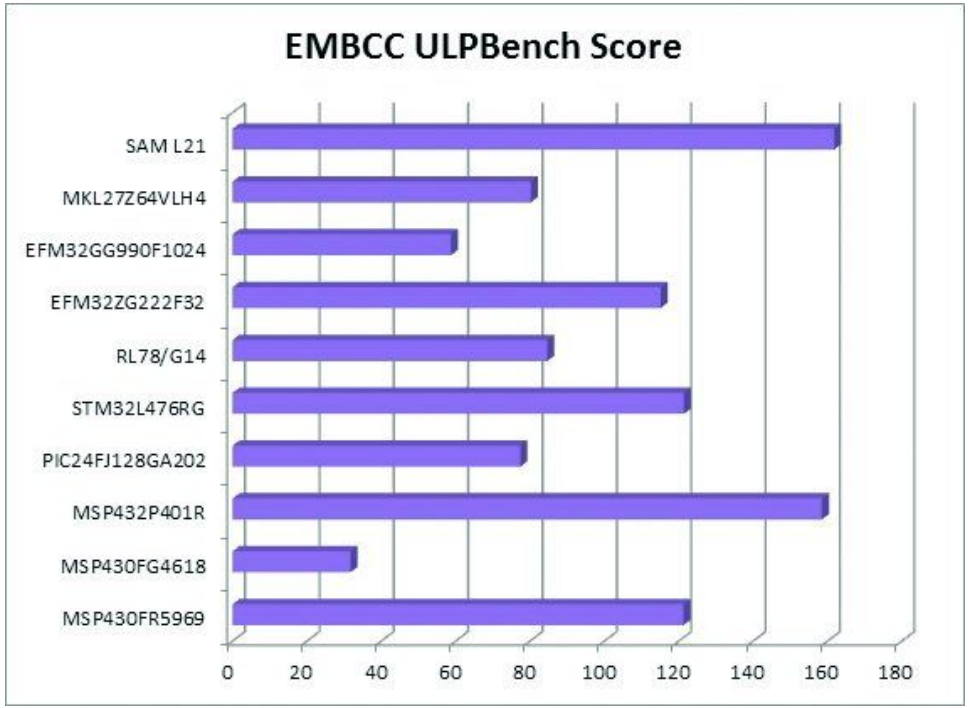
Benchmarks actuales: EEMBC y CoreMark

- Embedded Microprocessor Benchmark Consortium https://www.eembc.org/
  - *"EEMBC develops industry-standard benchmarks for the hardware and software used in autonomous driving, the Internet of Things, machine learning, and many other applications"*
- Por ejemplo:
  - **CoreMark** evalúa el CPU e incluye recorrer listas, operaciones matemáticas con matrices y procesamiento con máquinas de estado

Feature	Cortex-M0	Cortex-M0+	Cortex-M1	Cortex-M23	Cortex-M3	Cortex-M4	Cortex-M33	Cortex-M35P	Cortex-M55	Cortex-M7
Instruction Set Architecture	Armv6-M	Armv6-M	Armv6-M	Armv8-M Baseline	Armv7-M	Armv7-M	Armv8-M Mainline	Armv8-M Mainline	Armv8.1-M Mainline	Armv7-M
DMIPS/MHz*	0.87	0.95	0.8	0.98	1.25	1.25	1.5	1.5	1.6	2.14
CoreMark®/MHz*	2.33	2.46	1.85	2.64	3.34	3.42	4.02	4.02	4.2	5.01

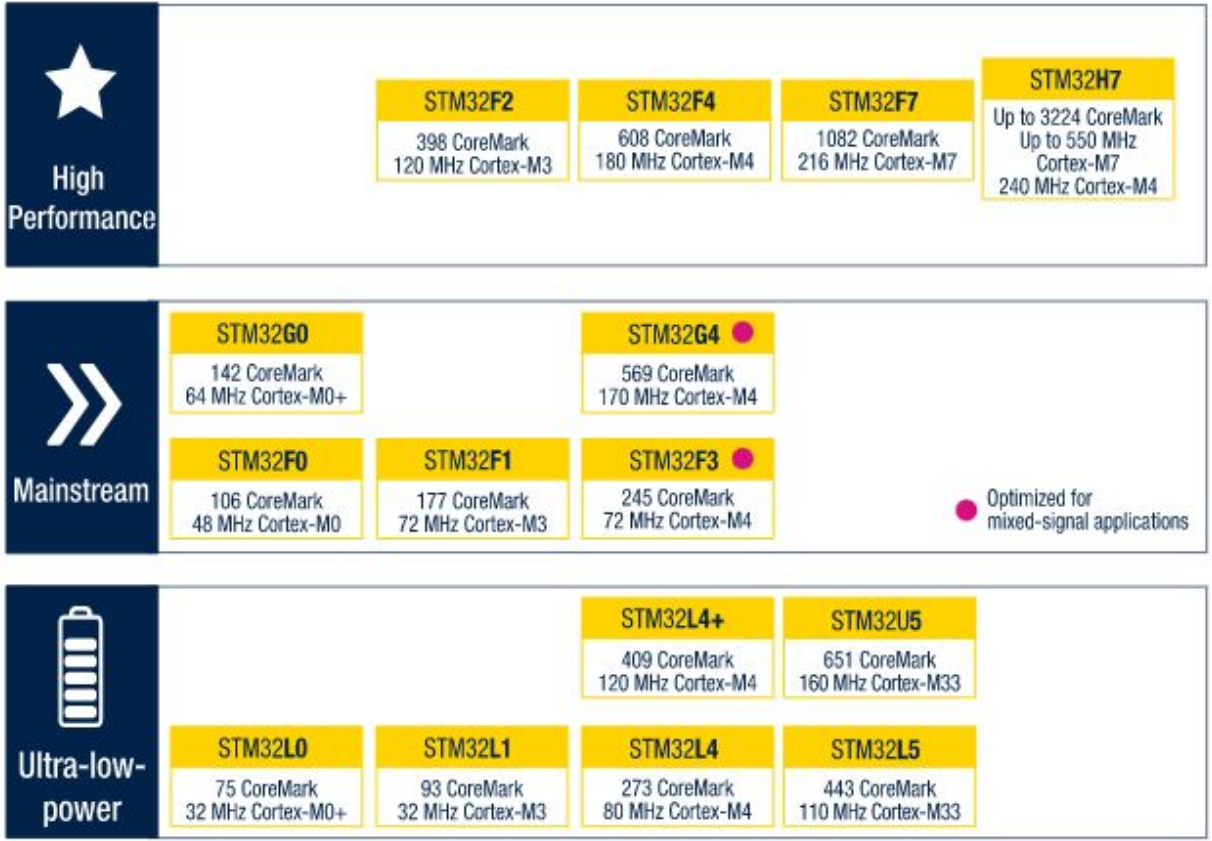
Benchmarks actuales: ULPBench

- **ULPBench**: Serie de tests que evalúan el consumo de energía apuntando a aplicaciones de consumo ultra-bajo.
- Se consider ULP si **dura más de 4 años** ejecutando una función activa que incluye una serie determinada de pruebas **1 vez por segundo** con una batería de **225 mAh**.
- Su primer test es Core Profile que mide la eficiencia del CPU, reloj de tiempo real, calendario, y modos de ahorro de energía.



https://www.eembc.org/techlit/articles/elektroniknet_article_2015_07_21/index.html

Asociado al microcontrolador



Algunas comparaciones

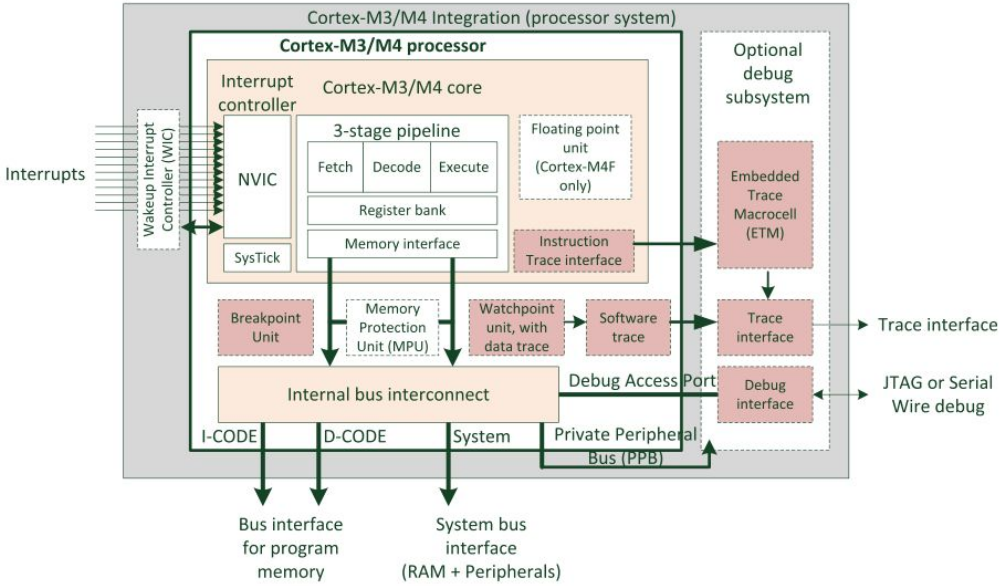
Empresa	µC	Compilador (influye!)	Memoria de ejecución	Tasa de clock [MHz]	CoreMark	CoreMark / MHz
Atmel	ATmega2560 (8-bit)	avr-gcc 4.3.2	Internal Flash	8	4.25	**0.53**
ST	STM32F103RB (Cortex-M3)	GCC 4.4.1	Internal Flash	72	108.26	**1.50**
Espressif	ESP32-S2 (Xtensa LX7)	GCC 8.4.0	Stack (DRAM)	240	472.81	**1.97**
ST	STM32H743XI (Cortex-M7)	GCC 9.3.1	Internal Flash	480	2434.6	**5.07**
SiFive	FE310-G002 (RISC-V RV32)	GCC 8.3.0	Internal Flash	320	515.20	**1.61**

https://www.eembc.org/coremark/scores.php

Arquitectura del procesador Cortex M3

Arquitectura del Cortex-M3

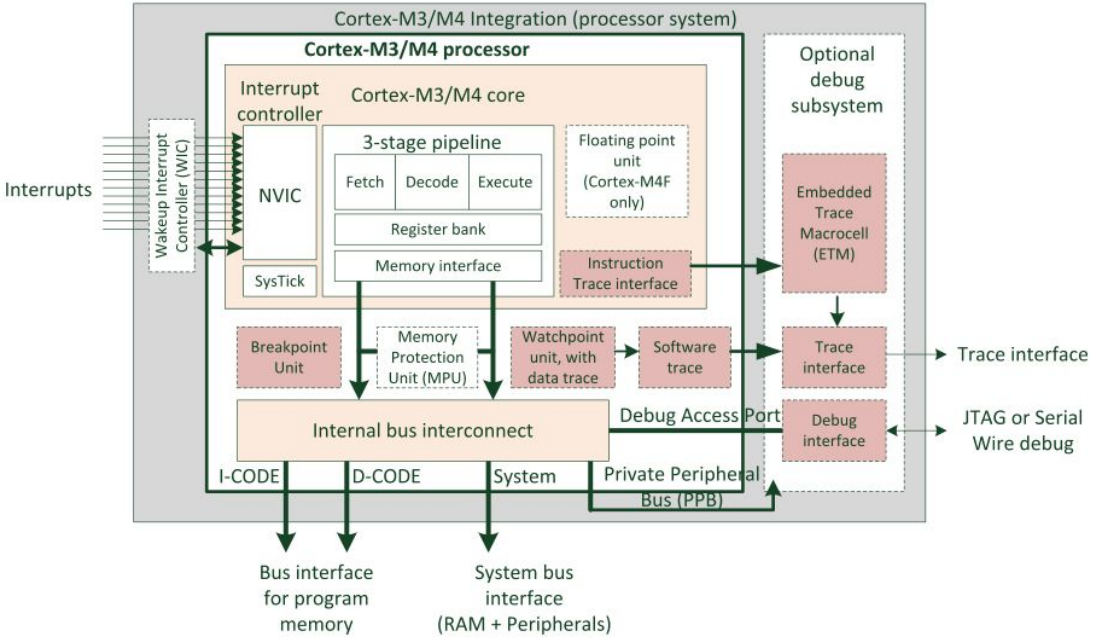
- Procesador de 32 bits
  - Es decir que sus registros internos y sus buses son de 32 bits
  - Por lo tanto puede direccionar una memoria de **4 GB**
- Arquitectura **Harvard modificada**
  - Accesos independientes a memoria de datos y código
  - Pero externamente se unen (aunque el bus permite accesos independientes)
- Pipeline de **3 etapas**
  - Permite un CPI bajo



Joseph Yiu - The Definitive Guide to ARM Cortex -M3 and Cortex-M4 Processors

Arquitectura del Cortex-M3

- Set de Instrucciones RISC
  - Aunque la “historia” de ARM lo hace más voluminoso
- En particular la arquitectura de instrucciones es **ARMv7-M**
- Es un set “reducido” de instrucciones que privilegia
  - Lograr un tamaño de código reducido
  - Operación determinística por sobre la performance
- Utiliza instrucciones Thumb 2 en su mayoría de 16 bits y que pueden ejecutarse casi todas en el equivalente de 1 ciclo de reloj
  - ¿Ventaja?



Joseph Yiu - The Definitive Guide to ARM Cortex -M3 and Cortex-M4 Processors

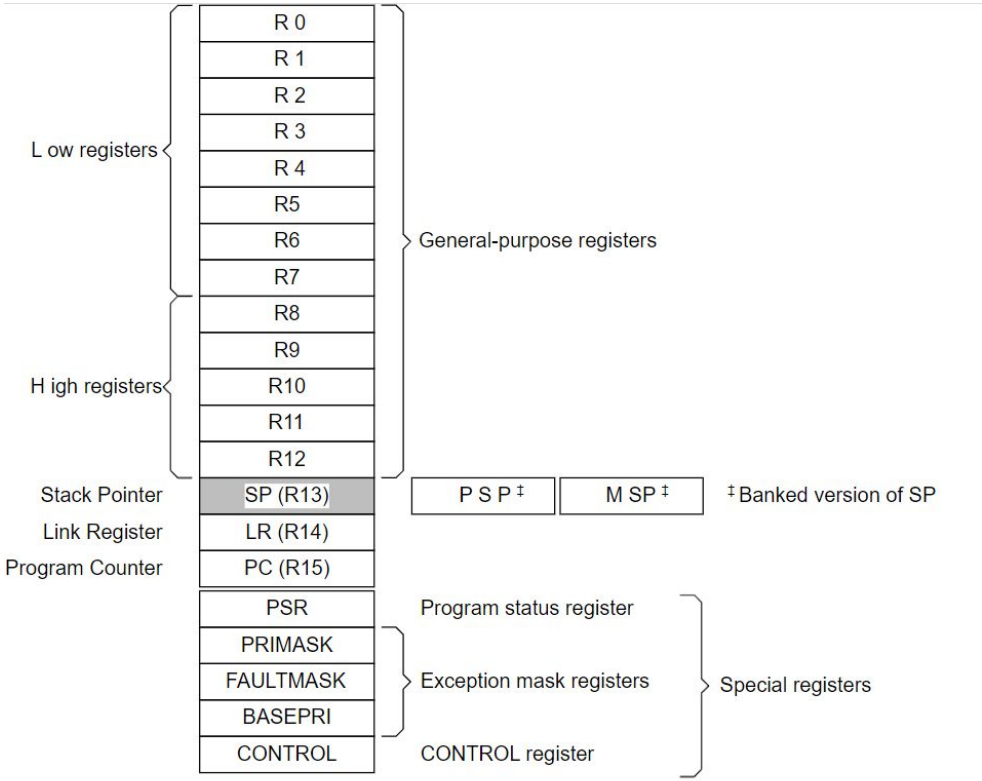
Registros

Registros del procesador

- Son el espacio “donde todo sucede”
- 16 registros de propósito general + regs. especiales
- Debido al set RISC se usa **arquitectura Load-Store**
- Todas las instrucciones operan sólo dentro de estos registros (excepto por supuesto load y store)

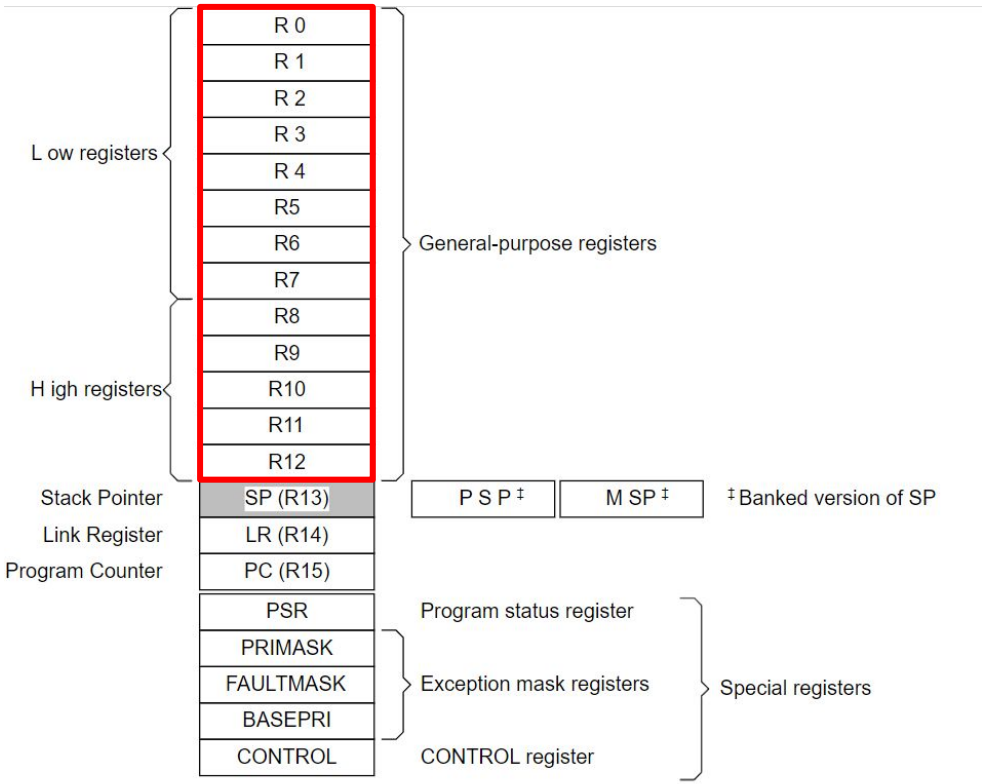
ARM	MSP430

`LDR R5, [R0];`
`ADD R5, R5, #10;`
`STR R5, [R0];`
	`ADD #10, 0(R5)`



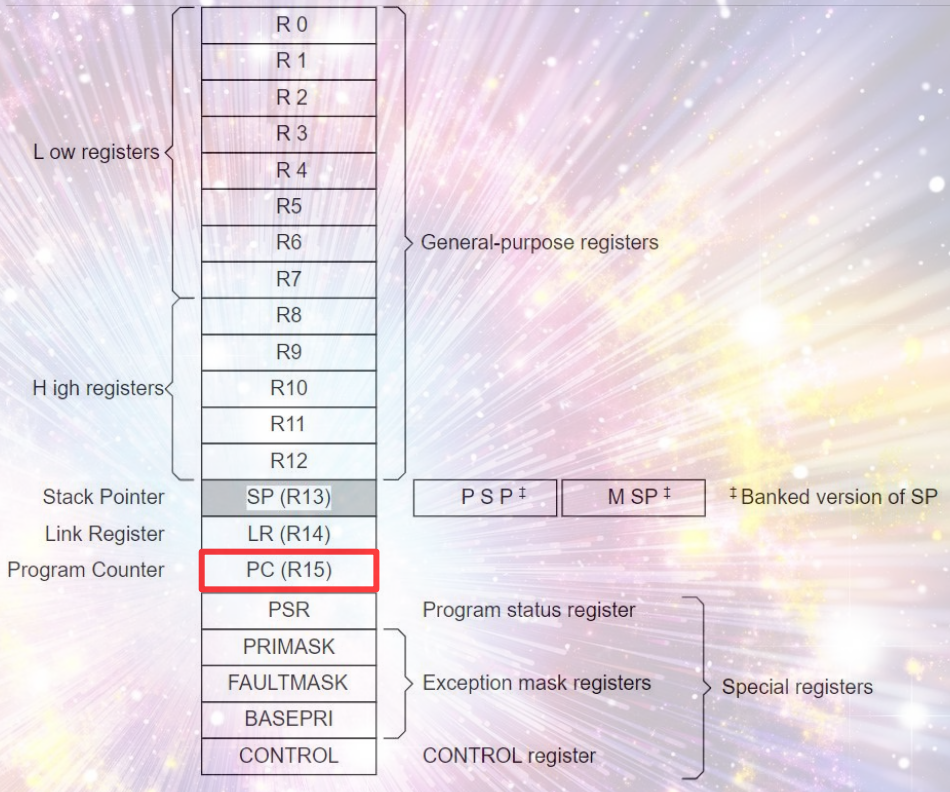
Registros de propósito general

- Se utilizan en todas las operaciones
- En contraste con arquitecturas más antiguas que tenían funciones especiales en registros (como el acumulador) dan más libertad al compilador
- A algunos registros se le asigna funcionalidad especial no por hardware si no por el estándar del compilador
  - R0 a R4 para pasaje de argumentos a subrutinas
  - R7 para puntero “modificable” al stack



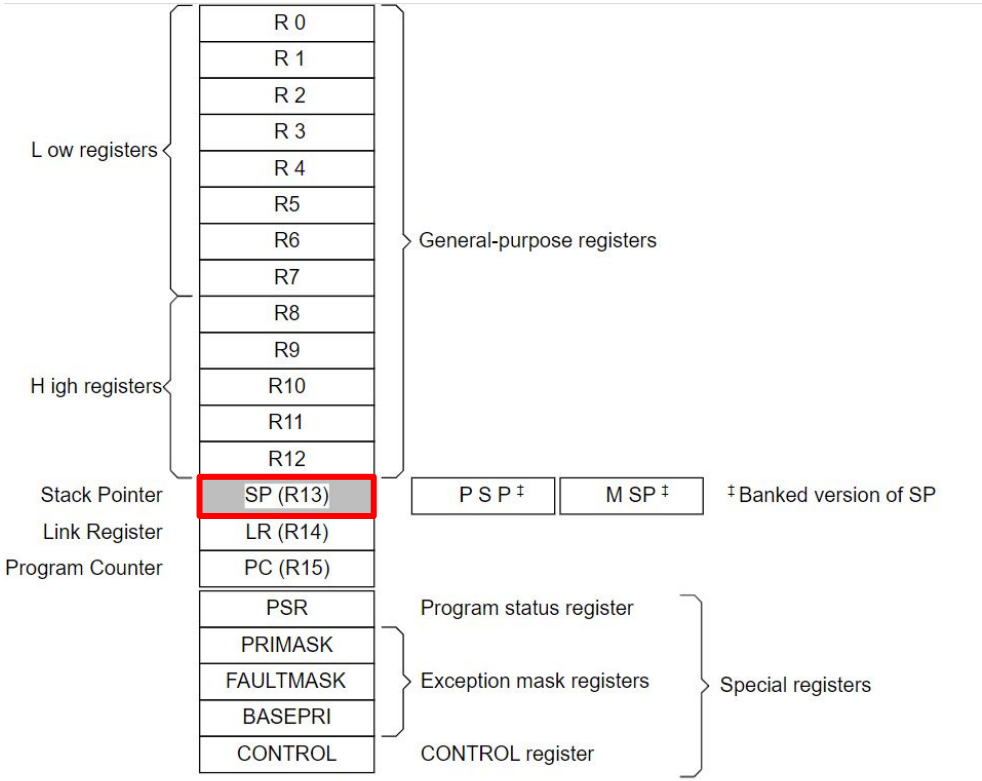
Program counter

- El PC mantiene la dirección de la instrucción a ejecutar.
- Se inicializa en la dir. 0x00000004 donde está la “tabla de vectores”
- La **tabla de vectores** es El Comienzo de Todo!
- ¿Por qué la 4? En la 0 está la dirección del stack
- Valor de las direcciones en el PC:
  - Se puede alinear a 32 o 16 bits.
  - Por lo tanto el LSB sería 0
  - Pero debe escribirse siempre a 1 para indicar “thumb state”



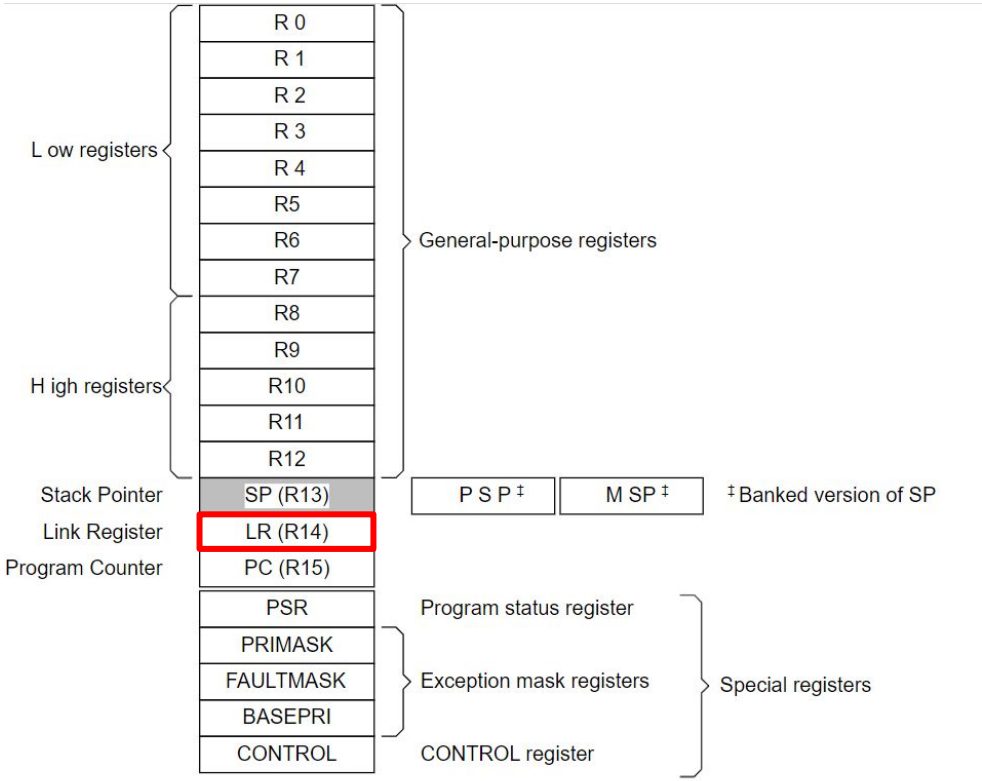
Stack pointer

- El stack es **la "memoria de trabajo"** del programa y el lugar donde guardar el contenido de los registros al **cambiar de contexto** (saltar a otra función).
- El SP mantiene la dirección del segmento de stack que se está usando y permite rápidamente a través de instrucciones push y pop que lo modifican automáticamente cambiar de contexto.
- Es **"full descending"**: Al iniciar la ejecución se apunta **a la posición más alta** de la memoria RAM y se decrementa al hacer los push.
- Tiene 2 "bancos" o registros físicos. Según el estado de ejecución muestra el "Main SP" o el "Process SP" (el PSP se usa con OSs)
- Los últimos 2 bits son siempre 0 (push y pop son siempre de 32 bits)



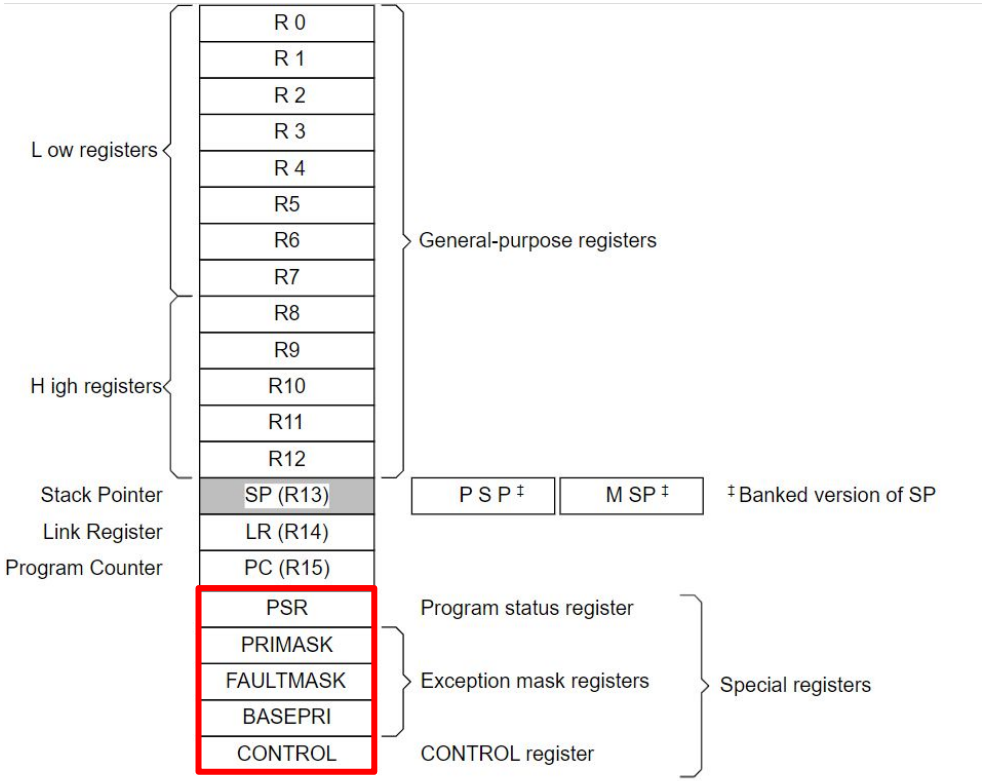
Link register

- Guarda el salto de retorno al cambiar de contexto con la instrucción “branch with link”
- Cuando se atiende una excepción almacena un código especial



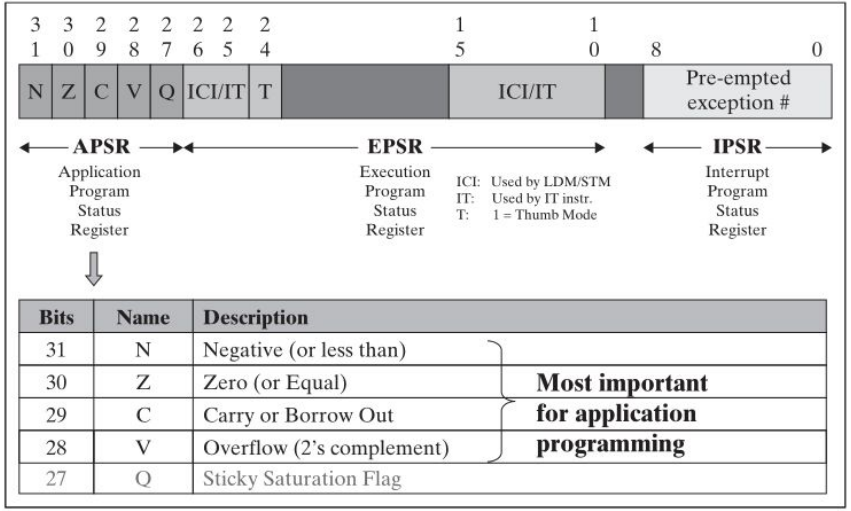
Registros Especiales

- Sirven para conocer y controlar el estado de ejecución del código
- Se accede con instrucciones
  - MRS <reg>, <special_reg>;
    - Read special register into register
  - MSR <special_reg>, <reg>;
    - write to special register
  - Estas instrucciones pueden “bloquearse” en un modo privilegiado y de esa manera se implementa un rudimentario control de acceso



Program Status Register

- Application PSR:
  - Contiene los resultados de las operaciones y es utilizado constantemente en cualquier aplicación (negativo, cero, carry, overflow, saturación)
- Execution PSR:
  - Le sirve al procesador para asentar el estado durante instrucciones que conllevan varios pasos secuenciales (load/store múltiple e if/then)
- Interrupt PSR
  - Se relaciona con el manejo de excepciones



Bits	Name	Description
31	N	Negative (or less than)
30	Z	Zero (or Equal)
29	C	Carry or Borrow Out
28	V	Overflow (2's complement)
27	Q	Sticky Saturation Flag

Most important for application programming

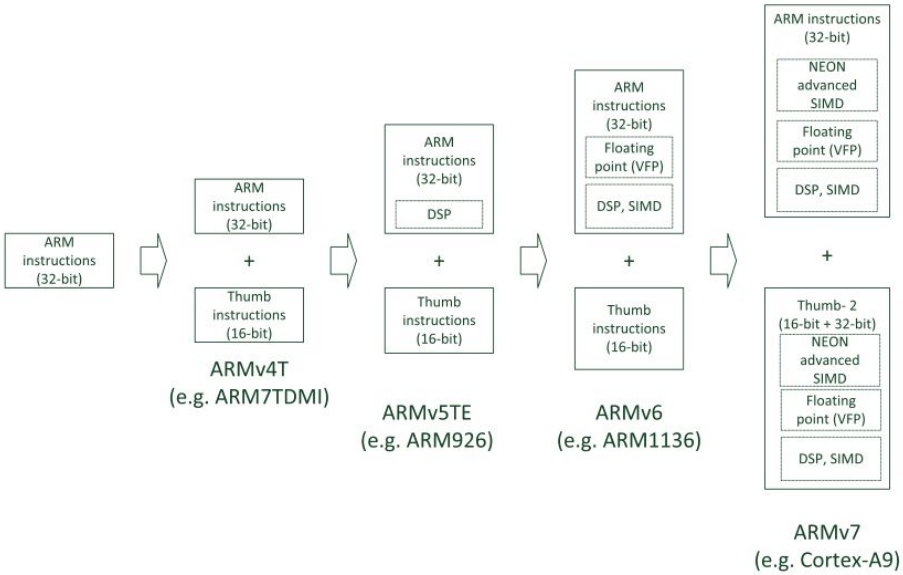
FIGURE 5-12 Bit fields within the Program Status Register.

D.W. Lewis - Fundamentals of Embedded Software with the Arm Cortex M3 (2012)

Set de Instrucciones

Set de Instrucciones thumb-2

- Instruction Set Architecture (ISA) **ARMv7-M**
  - Documentación: ARMv7-M Reference Manual
- Son de tipo Thumb 2, es decir que mezclan 16 y 32 bits.
- La arq. -M que privilegia tamaño y determinismo, **sólo opera en modo Thumb y prevalecen las instrucciones de 16 bits.** Por compatibilidad (o portabilidad) se mantiene el bit de modo thumb.
- Al ingresar al procesador, todas las instrucciones **se expanden a 32 bits**



Manejo de datos

- ISA ARMv7-M soporta los siguientes tipos en memoria:
  - **Byte** 8 bits
  - **Halfword** “media palabra” 16 bits
  - **Word** “palabra ”32 bits
- Existen instrucciones para los siguientes tipos de datos en los registros:
  - Punteros de 32 bits
  - Enteros con o sin signo de 32 bits
  - Enteros sin signo de 16 o 8 bits almacenados rellenando con 0s
  - Enteros con signo de 16 o 8 bits almacenados con extensión de signo
  - Enteros de 64 bits con o sin signo almacenados en 2 registros

Direccionamiento

- El direccionamiento de memoria en las instrucciones puede darse a través de varios métodos, entre ellos:
  - El contenido de registros (incluyendo **un shift opcional**)
  - Valores constantes
  - Valores relativos al PC (un método que el compilador usa comúnmente)
  - Valores relativos al SP
- El formato es **Little Endian** (el byte menos significativo en la dirección más baja de memoria o “primero”) y aunque puede ser configurable, es el modo más común y así lo encontraremos en la documentación de ST

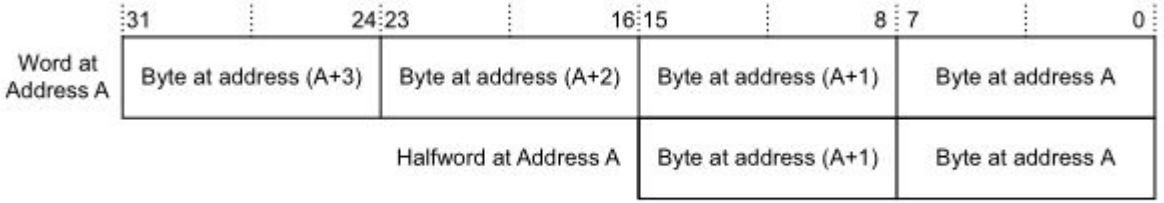


Figure A3-1 Little-endian byte format

Cortex M3 Reference Manual, ARM DDI 0403D

Operaciones y formato de las instrucciones

- El CPU puede realizar varias operaciones de bajo nivel en paralelo
- Existe un "**extensor de signo**" para expandir los operandos a 32 bits
- También **un barrel shifter que permite hacer el desplazamiento que se quiera en 1 ciclo** (que además puede ayudar a la ALU a realizar multiplicaciones). Algunas instrucciones pueden incluir desplazamientos.
- Un "memory address calculator" ayuda a calcular los direccionamientos y un sistema de incremento permite ir avanzando sin recalcular
- **Todas las instrucciones incluyen ejecución condicional** según los resultados del APSR
  - Esto puede ahorrar ciclos al evitar una instrucción de salto para evaluar una condición
  - Pero en arquitecturas más complejas con predictores de instrucciones y ejecución múltiple es contraproducente

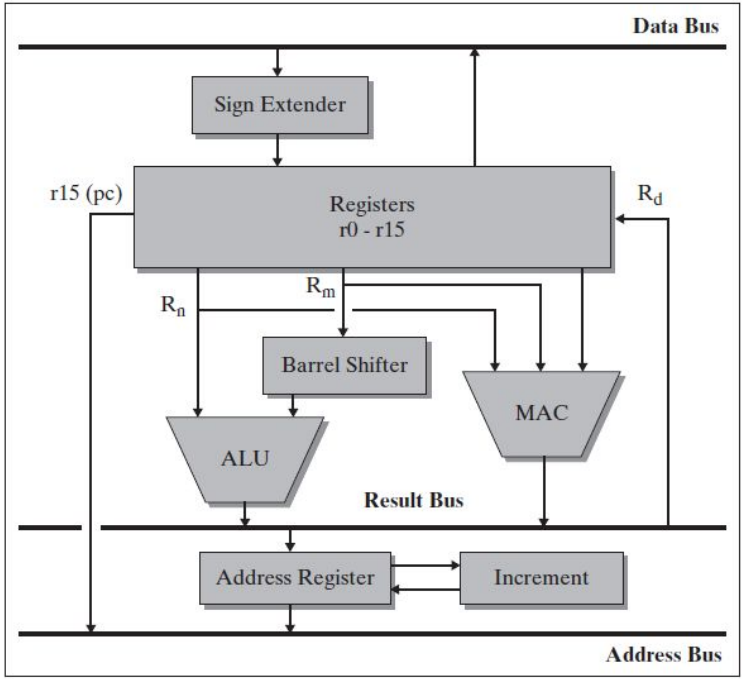


FIGURE 5-11 Internal organization of the ARM Cortex-M3 v7M CPU.

D.W. Lewis - Fundamentals of Embedded Software with the ARM Cortex M3 - Pearson (2012)

Impacto en el alto nivel

Barrel Shifter Integrado

- El desplazador (barrel shifter) permite realizar shifts y rotaciones en paralelo con otras instrucciones sin penalización en ciclos.
  - Optimiza operaciones como `x << n`, `x >> n`, `x & (1 << n)`, evitando ciclos adicionales.
  - Hace que cálculos como `x * 8` (equivalente a `x << 3`) sean eficientes.
  - Permite que algunas operaciones bit a bit se realicen en una sola instrucción ensamblador.

Expansor de Signo

- Un bloque de hardware que expande valores de 8 y 16 bits a 32 bits sin necesidad de instrucciones adicionales.
  - Hace eficientes las conversiones de tipos (`char -> int`, `short -> int`).
  - Evita el uso de instrucciones extra para expandir el signo.

```
int8_t a = -5;
int32_t b = a;  // Expansión de signo eficiente
```

Impacto en el alto nivel

Multiplicación por Hardware

- El Cortex-M3 tiene multiplicación de 32 bits por hardware en solo 1 ciclo.

```
int32_t resultado = a * b;  // Se ejecuta en un solo ciclo (más load y store)
```

Instrucciones de División por Hardware

- El Cortex-M3 tiene división entera por hardware, aunque toma más ciclos (2 a 12).
- Permite ejecutar divisiones en hardware sin librerías externas.

```
int32_t resultado = a / b;
```

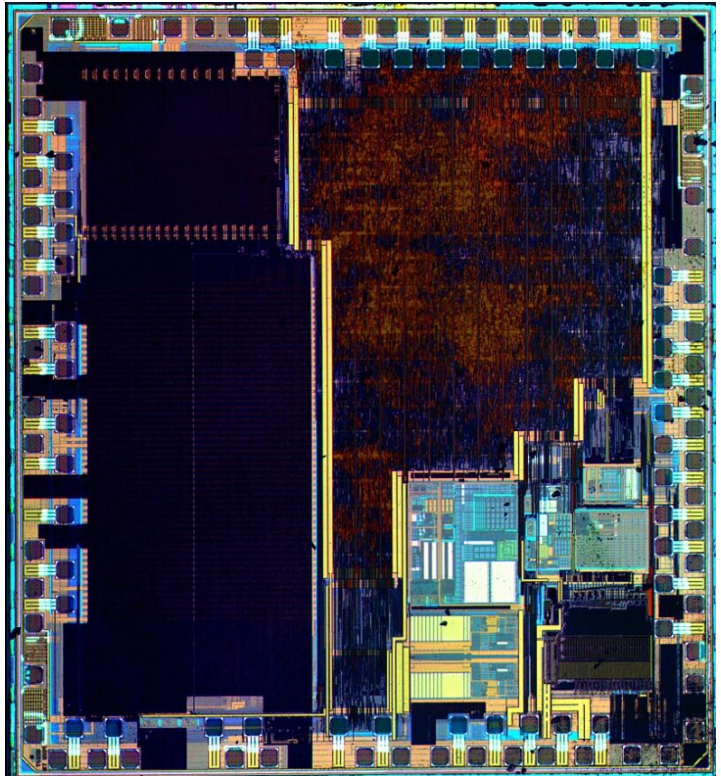
Instrucciones de Exclusión Mutua (LDREX/STREX)

- Instrucciones `LDREX` y `STREX` permiten operaciones atómicas en memoria.
  - Facilita la implementación eficiente de semáforos y locks

Comparación con otras arquitecturas

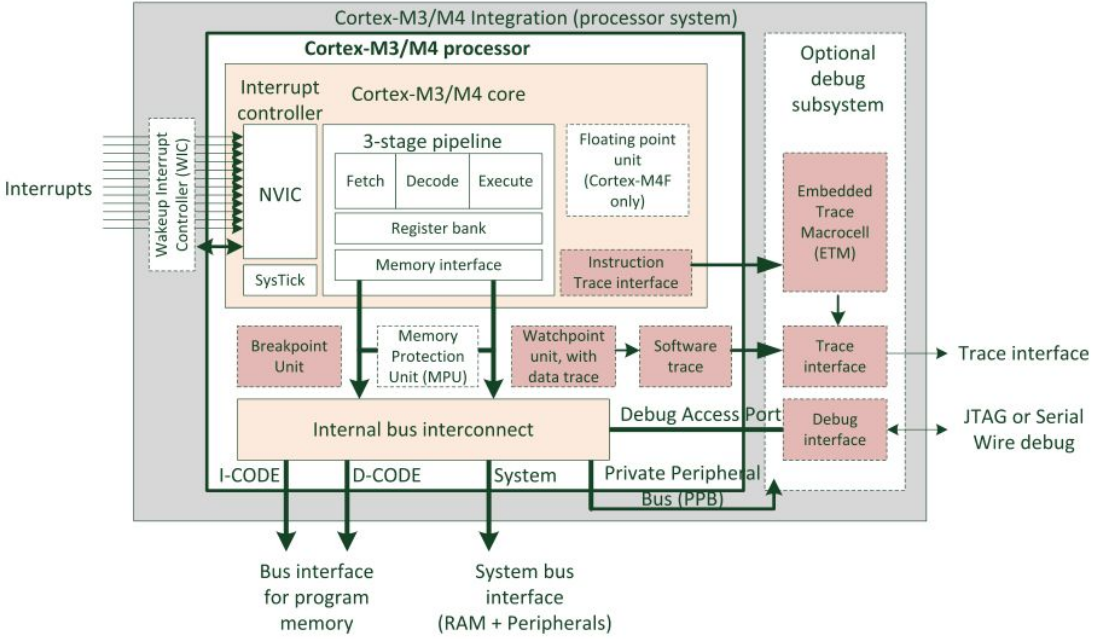
Característica	ARMv7-M (Cortex-M3)	RISC-V (RV32I/C)	Xtensa (LX6/LX7 - ESP32)
Registros	16 (R0-R15)	32 (x0-x31)	32 a 64 (usa "ventanas")
PC y SP	R15 (PC) y R13 (SP) son registros fijos.	x2 es SP (por convención). PC es especial.	Registros especiales fuera del banco principal.
Manejo de Cero	No existe registro cero físico.	x0 cableado a cero.	No existe registro cero físico.
Set de Instrucciones	Thumb-2 (16/32 bits)	Modular (32 bits + 16 bits C)	Variable (24 bits base + 16 bits opcionales).
Banderas (Flags)	Sí (NZCV).	No (comparación directa).	Sí (registros de estado).
Interrupciones	Gestión por Hardware (NVIC).	Gestión por Software (CSRs).	Gestión por Hardware + Software.

Memorias y modelo de memoria



Acceso a memoria y periféricos: Procesador

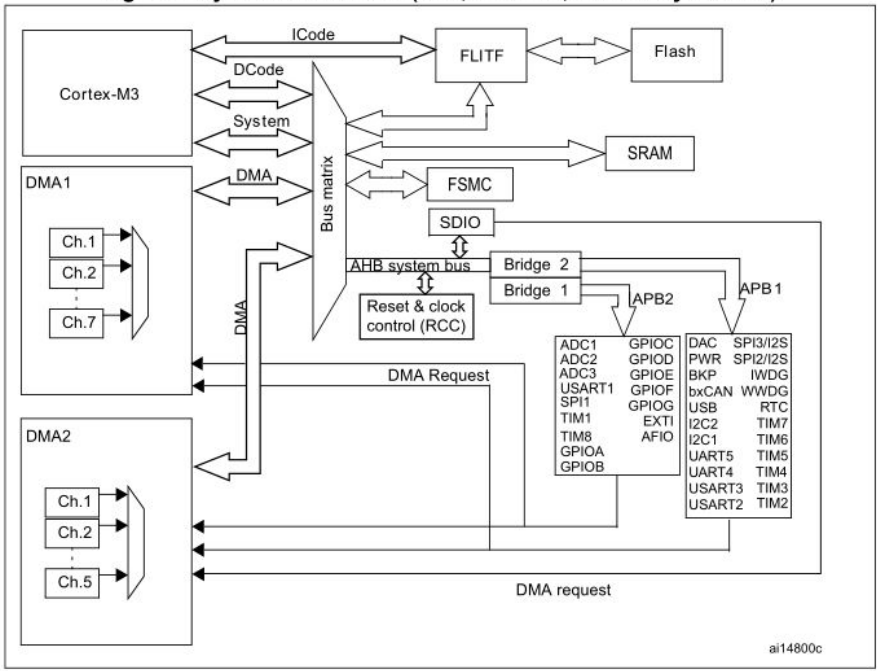
- Arquitectura Harvard modificada
- Buses que tiene disponible el procesador:
  - I-CODE
  - D-CODE
  - System
  - Debug Access Port
  - Private Periphereal



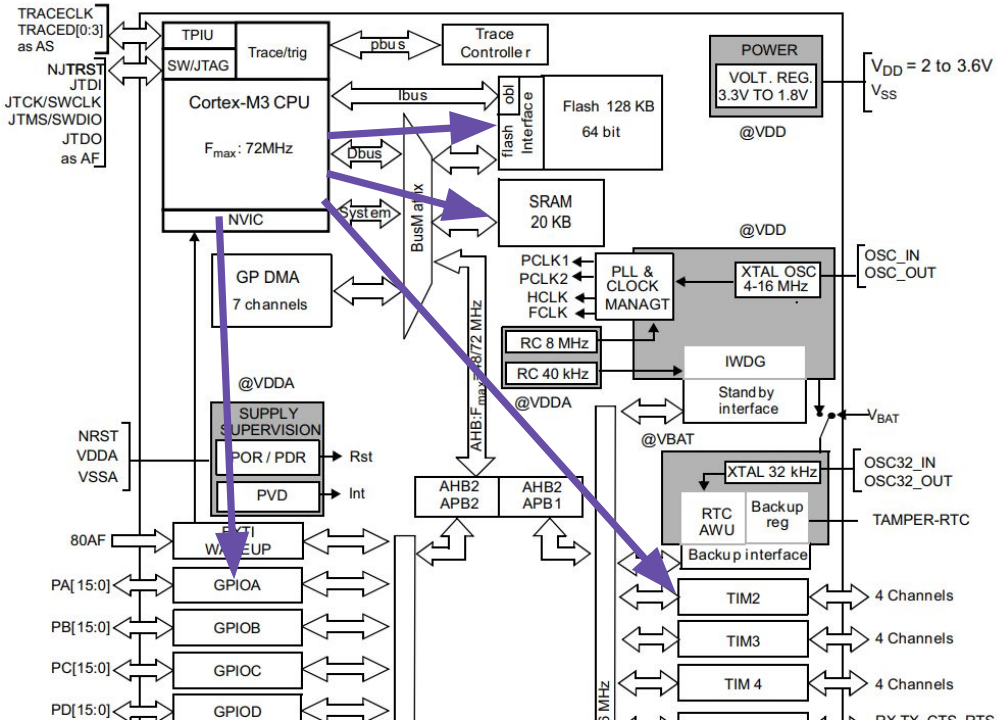
Acceso a memoria y periféricos: Microcontrolador

- Acceso a memoria de programa (Flash)
  - Por ICode para instrucciones, DCode para datos
  - FLIFT es un caché de prefetch de 2×64 bits, permite escribir y borrar la Flash de a 16 bytes (reprogramar)
  - Acceso a través del DAP a la memoria flash para la programación
- Acceso a la memoria de datos (RAM) y periféricos
  - **Periféricos en buses APB1, APB2**
- Un arbitrador da prioridad a los distintos maestros
  - Maestros: Cortex, DMA
  - Esclavos: Memorias, Periféricos

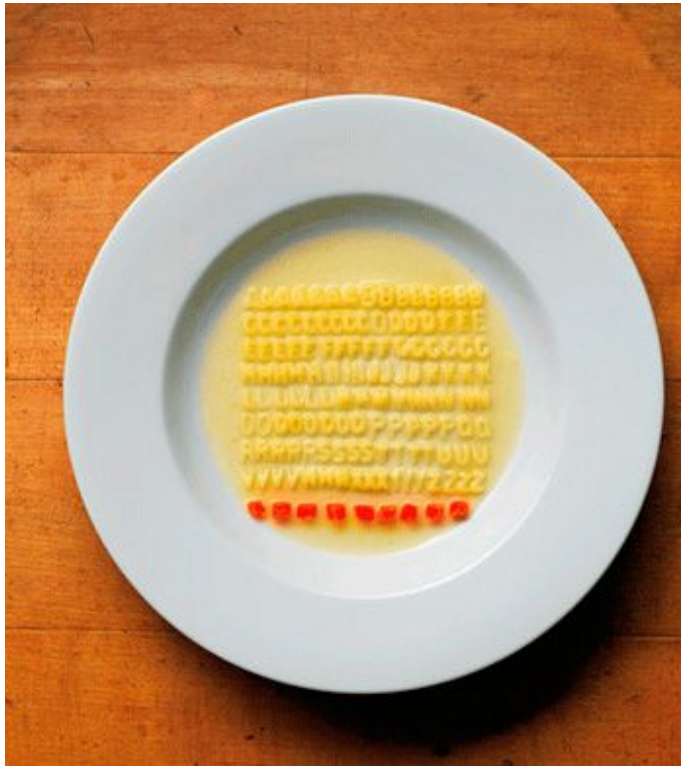
Figure 1. System architecture (low-, medium-, XL-density devices)



Vista del sistema desde el procesador



Vista del sistema desde el procesador: Modelo de memoria



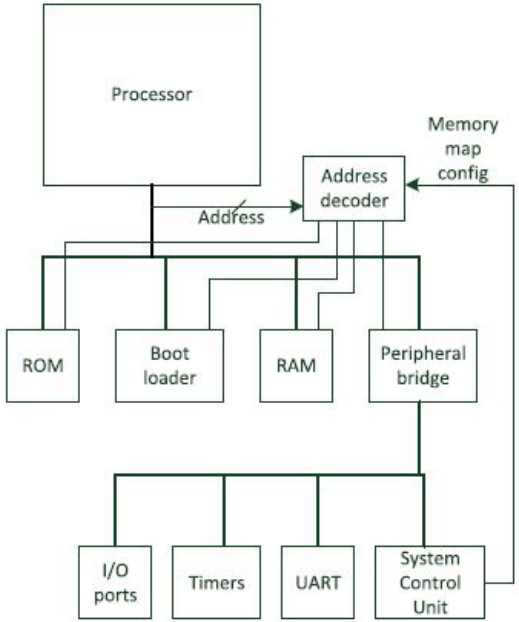


FIGURE 6.19

A simplified memory system with configurable memory map

Modelo de memoria - Cortex M

- Los procesadores Cortex-M ven **todo** como una posición de memoria **en un mapa lineal de 4 GB** (el máximo direccionable con 32 bits)
- No son direcciones “reales”, pero pueden verse como si lo fueran para usar el dispositivo
- Las direcciones de los segmentos son un estándar dentro de la familia Cortex-M para poder portar el código

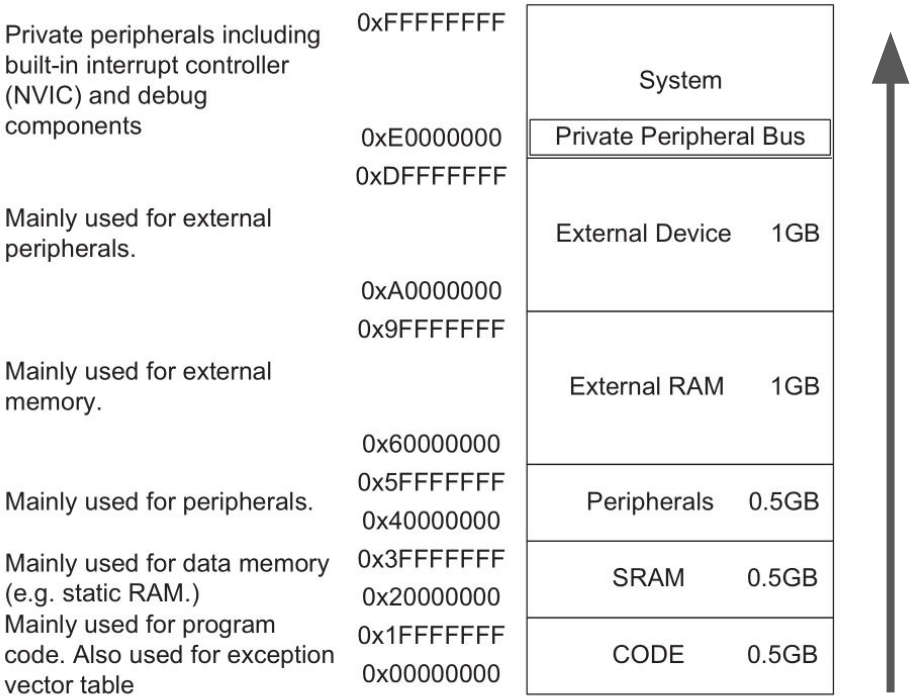
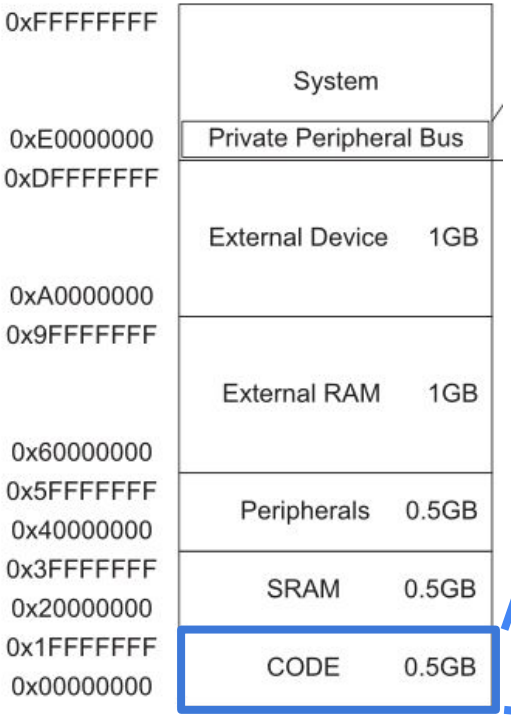


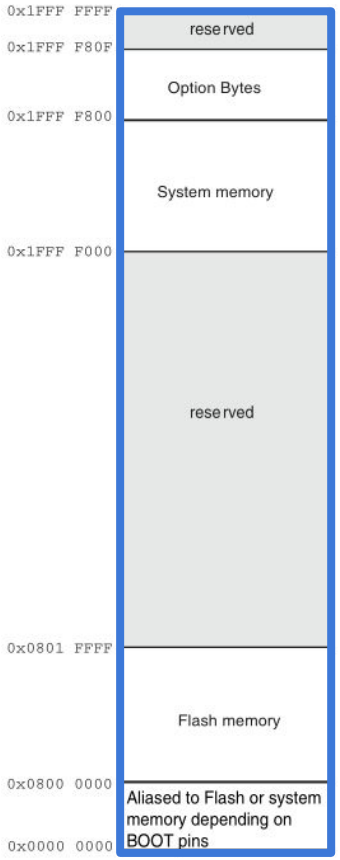
FIGURE 4.18

Memory map

Joseph Yiu - The Definitive Guide to ARM Cortex -M3 and Cortex-M4 Processors

Memoria de código





16 B — en FLASH (descriptos en PM0057)

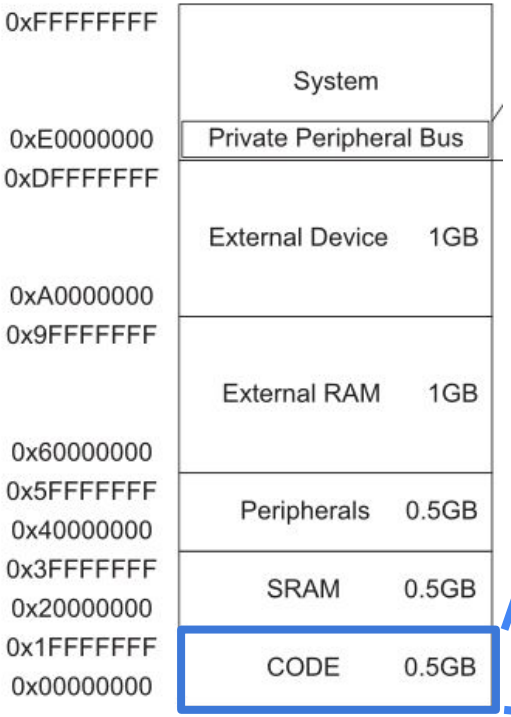
2 kB — en ROM

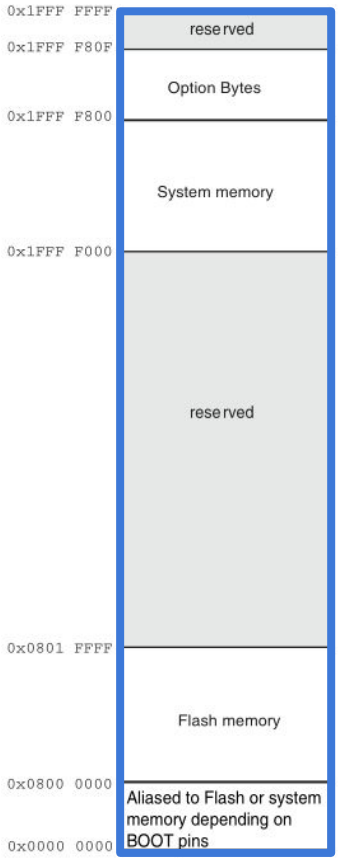
64 kB — en FLASH

no existe!

Datasheet - STM32F103x8 STM32F103xB
Doc ID 13587 Rev 12, ST

Memoria de código





Valores programables para controlar comportamiento al encender el dispositivo, relativo a protección de memoria, watchdog, eventos de reset para modos de bajo consumo

Bootloader pre-programado por ST en ROM con el código para reescribir la flash a través de algún puerto (e.g. UART)

Si el uC tiene 64 kB de Flash = 2^16 Bytes se utiliza desde la posición inicial X hasta la posición

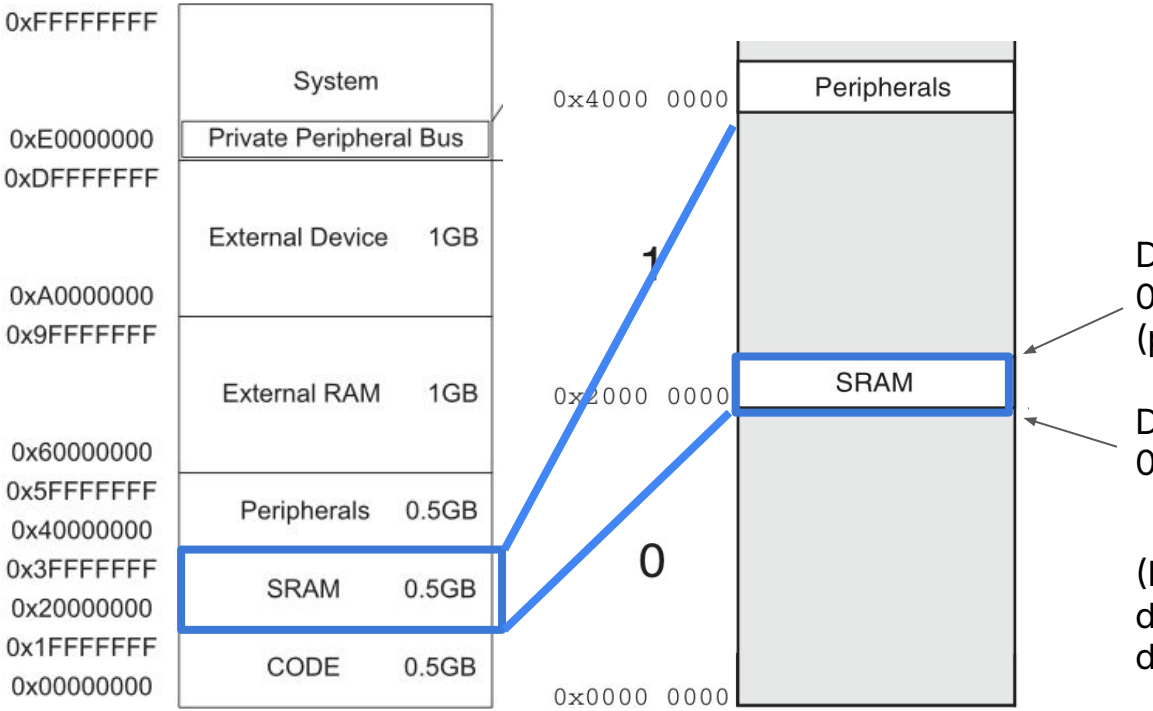
X + 0x10000 - 1 = A + 0xFFFF

del mapa de memoria (y equivalente para 128 kB)

Cuando se accede a la dirección 0x00000000 en realidad se redirige el acceso a la posición de la FLASH o del bloque System

Datasheet - STM32F103x8 STM32F103xB
Doc ID 13587 Rev 12, ST

SRAM



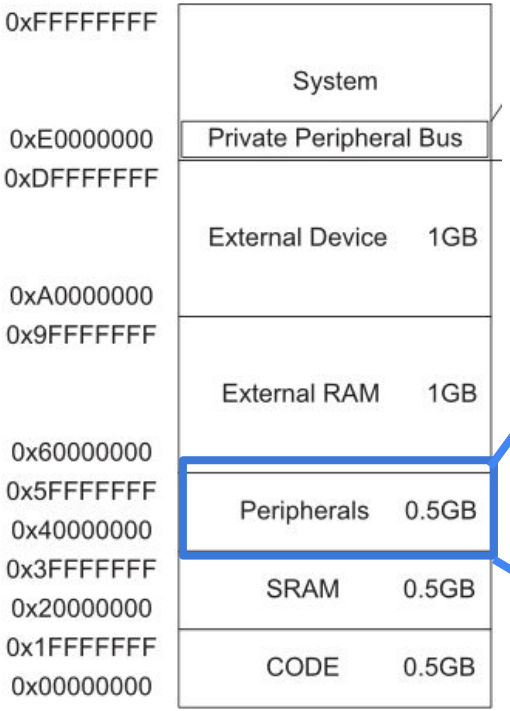
Dirección “inicial”
0x2000 5000
(para disp. de 20 kB)

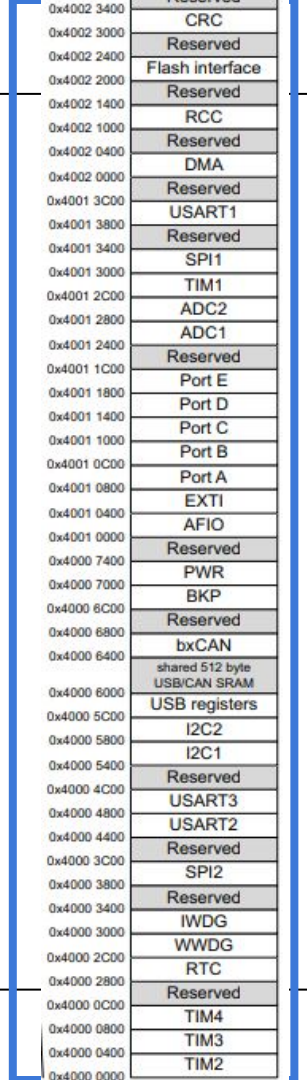
Dirección “final”
0x2000 0000

(Recordar que el stack pointer es “full descending”, es decir que se decrementa con cada push)

Datasheet - STM32F103x8 STM32F103xB
Doc ID 13587 Rev 12, ST

Periféricos

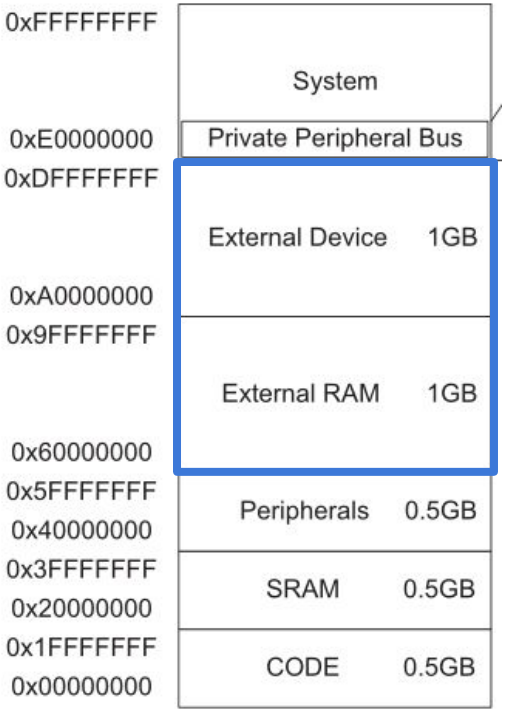




Joseph Yiu - The Definitive Guide to ARM Cortex -M3 and Cortex-M4 Processors

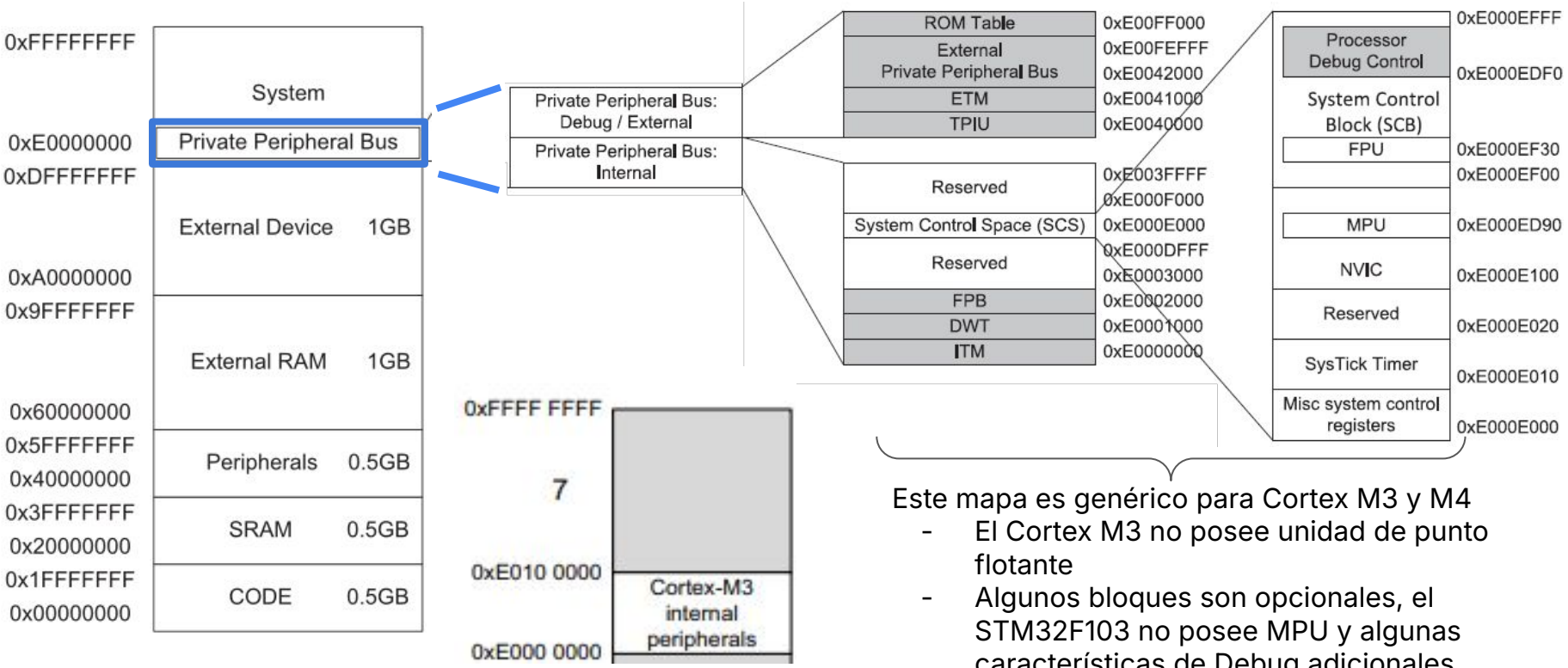
Datasheet - STM32F103x8 STM32F103xB Doc ID 13587 Rev 12, ST

Externos



Sirve para integrar memoria o periféricos externos en “igualdad de condiciones” con los internos. Requiere periféricos especializados (que el STM32f103 no tiene) y buses rápidos en general con un conteo de pines alto

Segmento privado del Cortex M3



Este mapa es genérico para Cortex M3 y M4

- El Cortex M3 no posee unidad de punto flotante
- Algunos bloques son opcionales, el STM32F103 no posee MPU y algunas características de Debug adicionales

Joseph Yiu - The Definitive Guide to ARM Cortex -M3 and Cortex-M4 Processors

Datasheet - STM32F103x8 STM32F103xB Doc ID 13587 Rev 12, ST

ARM DDI0337H Cortex M3 Technical Reference Manual (Revision r2p0), ARM